

JEST: REST on OpenJPA

Unifying the REST and JPA architectural styles for multitier web applications

Skill Level: Intermediate

[Pinaki Poddar](#)

Java Persistence Evangelist
IBM

01 Feb 2011

JEST unites two architectural styles — Representational State Transfer (REST) and the Java™ Persistence API (JPA) — to let language-neutral remote clients transact with JPA-based applications following REST principles. JEST models customizable persistent closures of managed entities as REST resources and represents them in XML or enhanced JavaScript Notation (JSON) format to account for the cycles in an object graph. This article explains JEST's concepts. Then it introduces a demonstration implementation: a generic web client that communicates with a server to query persistent objects and browse a persistent domain model in domain-agnostic fashion.

The Representational State Transfer (REST) architectural style defines how a client and a server can communicate to let multitier systems scale to a virtually unbounded number of clients. The Java Persistence API (JPA) defines how a server-side application should interact with persistent data stored in a relational database. REST and JPA share common architectural traits. This article describes and demonstrates JEST, a solution that unites these two styles into a robust, end-to-end architectural basis for multitier web applications. JEST is part of the Apache Software Foundation's OpenJPA project, which is an implementation of the Java Persistence 2.0 specification (see [Resources](#)).

JPA: More than an API

JPA is often misconstrued as nothing more than a solution for object-relational mapping (ORM), or as a mere replacement for Java Database Connectivity (JDBC). JPA, of course, is expressed

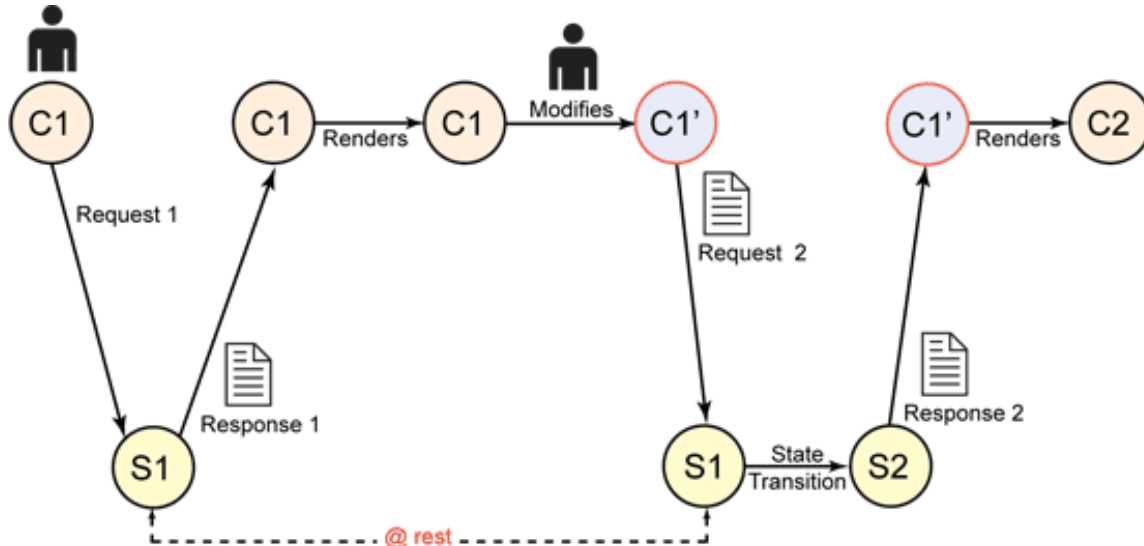
as an API in the Java language. But behind the API lies a model-view-controller (MVC) architectural style for object-oriented transactional applications.

This article begins by describing the salient features of REST and JPA to highlight their common traits. Then it elaborates on the technical and conceptual challenges to cooperation between the two styles and explains how JEST addresses them. The last section points you to a concrete implementation: a Java servlet that you can deploy in a servlet or application container and access via JEST's prepackaged Dojo/Ajax client from a vanilla web browser.

Principles of REST

REST principles have been widely adopted for the World Wide Web and cited as one of the primary reasons for its excellent scalability (see [Resources](#)). REST is based on a few key concepts that I'll expand on through the typical interaction scenario shown in Figure 1:

Figure 1. Example of REST-style interaction



- The interaction between a client and the server is solely based on a two-way transfer of *resources*. This is in contrast to language-bound, API-based interaction, whereby a client invokes server functions through remote proxies by passing variable arguments in a programming language.
- A *resource* is an identifiable, coherent concept. *Identifiable* means that every resource can be uniquely identified via well-defined syntax. The most common expression of such identity is a Uniform Resource Identifier (URI), such as a web address.

- The client requests a resource — an abstract notion such as the street address of a nearby coffee shop — from a server (for example, Google Maps). The server responds by sending a *representation* of the resource — for example, an HTML document with embedded JavaScript and a hyperlink to a map of the coffee shop's geographical location.
- A representation is *coherent* if it carries sufficient information to resolve all references contained within the representation.
- Upon receipt of the representation, the client often renders it in a browser. The noteworthy points are:
 - The abstract notion of a resource is realized as a concrete representation.
 - A server can respond with different representations for the same resource based on many criteria — for example, the user's natural language. The request can itself specify a particular acceptable form, such as a mime-type requested via the Accept header of a typical HTTP request.
 - A client may or may not resolve all the references in the representation. For example, a user may or may not decide to follow the hyperlink to display the geographical location map alongside the coffee-shop address, depending on the available real estate "on the glass."
- Besides rendering, the client can also allow the user to modify the representation or create new ones — for example, via an HTML form. After the user interaction completes, the client sends a new request along with the modified representation.
- The server receives the modified representation and, based on its content, mutates the original resource or creates new ones.

Listen: Podcast with Pinaki Poddar

In [this podcast](#), Pinaki Poddar offers detailed insight on the issues surrounding persistence in Java web applications. As a member of the Expert Groups for both JPA and JDO, and a committer for the Apache OpenJPA project, Dr. Poddar is well positioned to provide you with the latest information on this critical layer of any production-level web application.

An interaction between a client and a server via exchange of resource representations qualifies as REST-compliant *if* — and this is an important condition — the server is at *rest* after it sends the first response and before it receives the next request. That is, the server expends no computing power or other resources to hold the memory of the conversation with the client in between successive requests. (By

the same token, applications that store session data in the server are *not* REST-compliant.) Because the server does not maintain any client's context in between requests, the server's response can scale favorably against a virtually unbounded number of clients.

ACID transactions and REST

A stateless server is more realistic in practice when the client is only requesting resources. If the client sends back modified representations over multiple requests that the server must apply to resources *atomically* — that is, with an ACID (Atomicity, Consistency, Isolation, and Durability) transactional warranty — then compliance with the REST style is a challenging proposition. That is because an ACID transaction, by its very nature, requires both short-term and long-term memory.

An ACID transaction:

- Groups multiple client actions over a span of time into an atomic (indivisible) unit.
- Ensures that the combined effect of these actions is consistent.
- Ensures that the combined effect is isolated from any simultaneous actions by other clients.
- Makes the combined effect durable forever.

Short-term memory holds the current series of actions (also referred as *unit of work* or *in-flight transaction*). Long-term memory, often persistent and shared — such as a database on a hard disk — is required to make the combined effect durable.

So the key questions for an architect of a REST-style, client-server application are:

- What are the identifiable resources that a server offers?
- What are the resource representations that the server generates?
- How can a stateless server support ACID transactions?

Answers to these questions can be found within the context of a JPA-based server application catering to language-neutral web clients or web services without violating the core REST principles.

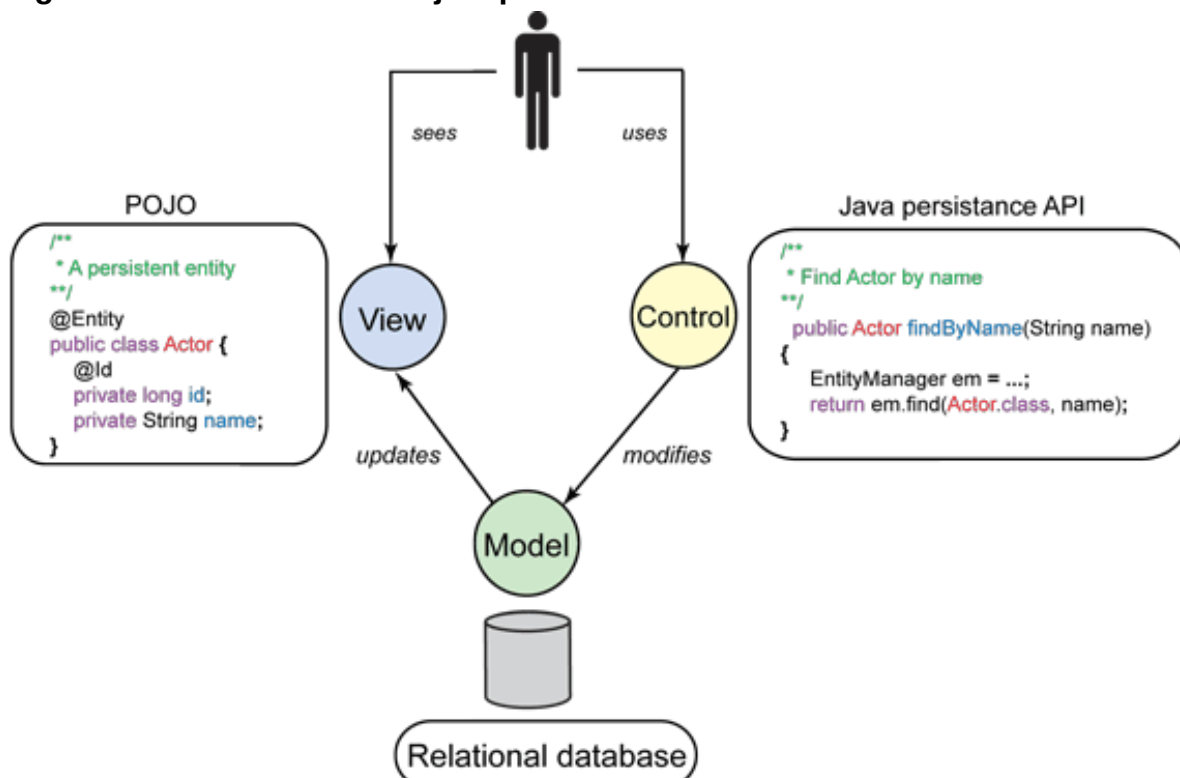
JPA: Model-View-Controller for object persistence

As an architectural style, JPA adheres to the classic Model-View-Controller (MVC) architecture for persistent Java objects:

- The model is a relational database.
- The view is a strongly typed Plain Old Java Object (POJO) without any *de jure* interface or superclass (as was the case for earlier Enterprise JavaBeans specifications).
- The controller is the API itself — that is, `javax.persistence.EntityManager`.

Figure 2 shows JPA's MVC architecture for object persistence:

Figure 2. JPA as MVC for object persistence



As [Figure 2](#) illustrates, a JPA-based application accesses the model (the relational database) only via the view (POJOs). All database modifications, such as inserting new records or updating existing ones, should be done only via the controller — by `EntityManager.persist()` or `merge()`, for example.

Applications that switch to JPA for persistence services gain an immediate advantage by offloading to a JPA provider the so-called ORM problem — the complex problem of converting between the database realm controlled by relational schema and SQL, and the object-oriented Java language realm ruled by strong typing and unidirectional references.

Why JPA >> ORM + JDBC

But the core proposition of JPA as a middleware component goes far beyond solving the ORM problem. Most relevant to this article's discussion is one of JPA's more powerful abilities: support for *detached transactions*.

A JPA provider guarantees ACID transactions comprising a series of persistent operations over a period T without holding a database connection for the entire duration of T . By holding the in-flight transaction in memory and narrowing the time window for a database connection, JPA makes the application more scalable against simultaneous user transactions. (It also potentially lowers costs, because the cost of a commercial database server is proportional to c^n , where c is the number of simultaneous connections allowed by a database installation and n is often much greater than 1.)

But the notion of detached transactions in JPA goes even further. JPA supports access-detach-modify-merge style transactions for remote clients. A client can request persistent objects from the server, and the server sends them as a graph of detached objects. The client can modify the detached object graph's content or structure and send the modified object graph back to the server in a separate request. The server can commit the modifications to the database with an ACID-compliant transaction warranty. The JPA application holds no reference to the detached objects between the first access request and the subsequent change request.

Such detached transaction semantics is even more significant for typical web-based clients for which the time span of the user transaction can be significantly longer than the actual database transaction, or when most of the transactions are read-only (that is, they never commit at all, the so-called *buy-to-browse ratio* typically being 1:200 for many popular websites). This access-detach-modify-merge can enable a JPA-based application to become scalable and cost-effective. Such an application can support, say, 5,000 simultaneous web clients with only 10 database connections without losing any transactions or lowering response time.

However, the access-detach-modify-merge programming model must satisfy certain prerequisites for client-server interaction:

- The client must be written in the Java language to invoke JPA through a remote session bean or similar facade.
- The server sends the detached object graph as serialized bytes of strongly typed persistent Java objects defined on the server side. To interpret these bytes, a client requires the class definitions of the persistent Java types.
- In practice, most JPA vendors introduce dependency on their implementation classes (such as proxies for standard Java collection types) into the user-defined POJOs to manage them more effectively.

That makes the client dependent on the runtime vendor libraries as well.

In contrast, JEST targets language-neutral clients and makes minimal assumptions about the client's computing environment, the lowest common denominator being a plain vanilla web browser. Hence, JEST must generate a representation for detached object graphs that are consumable by a language-neutral client, thereby unifying REST principles with JPA's access-detach-modify-merge programming model.

How JEST unifies REST and JPA

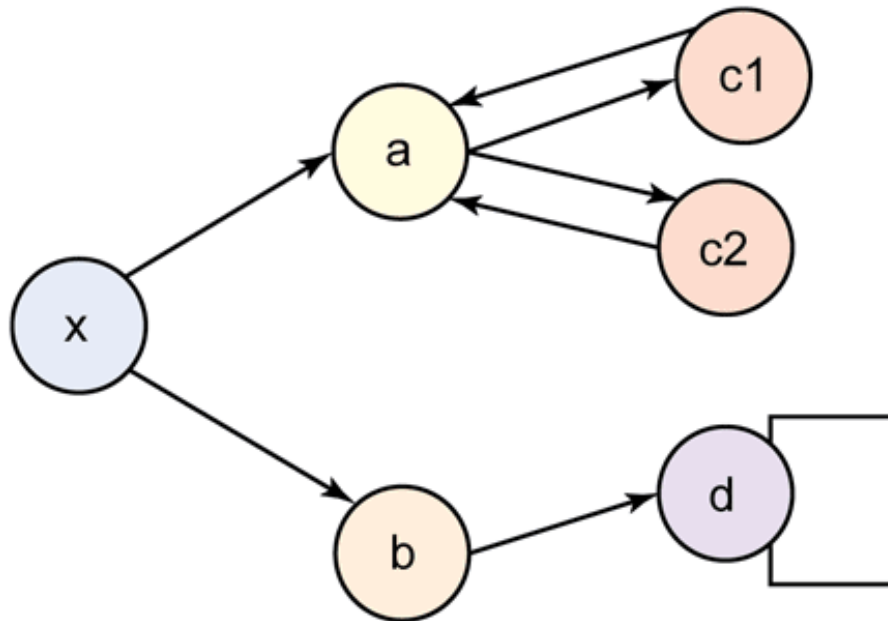
The key technical challenges of unifying REST and JPA, then, are:

- How to represent detached persistent object graphs as resources for language-neutral clients
- How to support ACID transactions without sacrificing the stateless nature of a RESTful server

JEST resources: Customizable, persistent closures

In the JEST context, the resource — the central concept of a RESTful architecture — is a customizable persistent closure of an identifiable, managed entity, as illustrated in Figure 3:

Figure 3. Customizable, persistent closure



$$C(x) = \{x, a, b, c1, c2, d\}$$

OpenJPA FetchPlan can customize

$$C(x) = \{x, a, b, d\} \quad \text{or} \quad \{x, b\}$$

A persistent closure of a managed root entity x is defined as the set of managed entities $C(x) = \{e\}$, where e is reachable from x directly or indirectly via a persistent relationship. Also, an entity is always reachable to itself, by definition.

Customization of a persistent closure is the ability to configure which entities are reachable from a root entity dynamically, at run time.

JPA prescribes that object relationships be decorated with persistence qualities. For example, a multivalued relationship between a `Department` and its `Employees` — that is, a private field in the `Department` class declared as `List<Employee> employees` — can be decorated as one-to-many relationship via the `@OneToMany` annotation or an accompanying mapping descriptor. In such a case, all `Employee` instances of a `Department d` become reachable from d . The closure can include indirect relation paths as well. If each `Employee e` has a reference to an `Address` instance a annotated with `@OneToOne`, then the `Address` instance a is transitively reachable from the `Department` instance via `Employee e`, and thus the closure of d includes an `Address` instance a as well.

However, it is important to note that persistent closure applies only to *managed* entities. So, for example, a `Department` instance `d` may be logically related to a set of 20 `Employee` instances — but that relationship might not have been loaded. That is, the `Employee` instances may not have been fetched from the database into the persistence context that holds `d`. In such a case, the closure of `d` won't include the `Employee` instances, nor will these instances be loaded from the database as a side-effect of evaluating the closure.

The current JPA specification provides some elementary facilities for the customization of closures. Any persistent property (relation as well as basic fields) can be decorated as `FetchType.LAZY` or `FetchType.EAGER`. Such decorations take effect when a root entity instance is being loaded from the database into a persistent context to determine which properties or other instances should be loaded as well. The major limitation of the current JPA specification is that such customization is defined statically, at design time. It is not possible to configure the closure at run time.

OpenJPA supports dynamic configuration of persistent closures through its `FetchPlan` interface (see [Resources](#)). `FetchPlan`'s rich syntax and semantics allow the application to configure the closure for any root instance resulting from a `find()` operation or instances selected by query. JEST leverages these mechanics to access a related set of entities and their properties. A fetch plan is a useful construct because the client can control the content of the requested representation for the same logical resource on a per-use-case basis. For example, the client can customize a representation it receives for the same coffee shop based on whether the information is rendered on a mobile phone or on a high-end desktop monitor.

Coherent JEST representations: XML and JSON

The next aspect of JEST's unification scheme is the resource representation. The representation is targeted for language-neutral, domain-agnostic clients. JEST prescribes the following qualities for resource representations:

JEST versus JAX-RS

Contrast the first constraint with the Java API for RESTful Web Services (JAX-RS). Under JAX-RS, persistent POJOs must be annotated in order to be represented. Because persistent POJO classes are part of the compilation unit, users are forced to recompile their applications. Moreover, such annotations are (as with JPA) defined statically, as `Fetch.LAZY` or `Fetch.EAGER`, and are not useful for customizing the contour of a closure on a per-use-case basis.

- **Noninvasive:** The resource must not require annotation or decoration of any kind in order to be represented (see the [JEST versus JAX-RS](#) sidebar).

- **Language-neutral:** The representation must be consumable by a client based on any programming language. Because the interaction is based on a URI rather than an API, a client written in any language can receive a representation via an HTTP request. The representation also follows a well-known format, such as XML or JSON, to enable any client to interpret its content. No client requires a plug-in to interpret a JEST-generated representation.
- **Domain-agnostic:** Even though the original resource is a strongly typed persistent Java object, the client does not require the persistent class definitions in order to consume the JEST-generated representation. In the REST sense, the representation is *self-describing*.

Domain-driven versus domain-invariant representations

A language-neutral representation is a ordered sequence of bytes or characters with an implied format. Common examples are XML and JSON. A string-based representation of this nature caters to a wide variety of clients by not assuming any special capability of the client other than its ability to parse a sequence of bytes or characters from a character set. JEST generates XML and JSON representations, extended (in the case of JSON) with a few essential capabilities.

The core theme of a JEST representation is that it is model-driven, as opposed to domain-driven. It is common to find XML representations for Java types that are domain-driven. For example, consider the simple persistent Java type in Listing 1:

Listing 1. Simple persistent Java type

```
@javax.persistence.Entity
public class Actor {
    @javax.persistence.Id
    private long id;
    private String name;
    private java.util.Date dob;
}
```

Listing 2 shows the corresponding XML representation of an Actor instance:

Listing 2. Domain-driven XML schema representing an Actor instance

```
<?xml version="1.0" encoding="UTF-8"?>
<Actor>
  <id type="long">1234</id>
  <name type="string">Robert De Niro</name>
  <dob type="date">Aug 15, 1940</dob>
</Actor>
```

Listing 2's XML representation is domain-driven in the sense that its implicit or explicit schema is governed by the type structure of the persistent domain class:

Actor. In contrast, a model-driven representation of the same instance looks like Listing 3:

Listing 3. Simple entity represented in a model-driven XML schema

```
<?xml version="1.0" encoding="UTF-8"?>
<instances>
  <instance type="Actor" id="Actor-1234">
    <id name="id" type="long">1234</id>
    <basic name="name" type="string">Robert De Niro</name>
    <basic name="dob" type="date">Aug 15, 1940</dob>
  </instance>
</instances>
```

A model-driven representation has a domain-invariant schema: it is generic enough to express any persistent type. In the model-driven representation, the schema for Actor does not vary, whereas for a domain-driven representation, it does. The advantage of a model-driven representation is that it allows the recipient to interpret the representation in a generic way that is independent of the received information's exact structure.

The model-driven schema named `jest-instance.xsd` is directly derived from the Metamodel API introduced in JPA 2.0 (see [Resources](#)). The Metamodel API is similar to the Java Reflection API for Java language types, extended for persistent types to gain more expressiveness. JEST expresses the JPA Metamodel constructs such as (so called meta-metadata) `javax.persistence.metamodel.Entity` or `SingularAttribute` etc. in an XML schema. All JEST responses for any persistent entity adhere to the same `jest-instance.xsd` schema.

Coherent representation by persistent closure

A persistent closure is a closed set. That is, for any e_1 in a closure $C(x)$, this holds true:

If e_1 refers to an entity e_2 , then e_2 must be in the same closure $C(x)$.

This *closeness* property ensures that a representation of a JEST resource is coherent — an important aspect of REST-style representation. The client receives a representation in which all references are resolved within the same representation, and the client does not need to make another server request to resolve any references.

Why JEST extends JSON

For XML representation, JEST defines its domain-invariant XML schemas for identity and references as `xsd:ID` and `xsd:IDREF` types, respectively. Hence, a standard XML parser can resolve the references to appropriate Document Object Model

(DOM) elements.

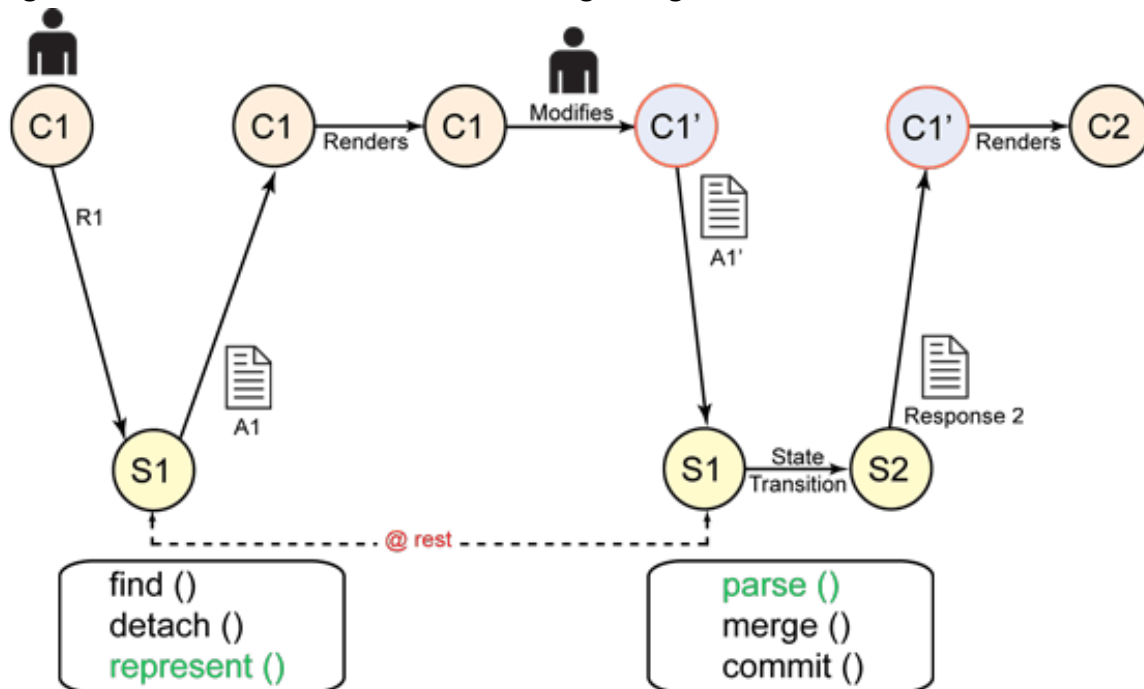
A coherent representation with JSON, however, faces a technical problem. Standard JSON encoding libraries, popular in web browser environments, do not support circular references (see [Resources](#)). (This is a surprising fact, given that circular references are a common occurrence for almost any persistent object graph.) To address this limitation of existing JSON encoders and, more important, to support the core premise of the representation being coherent to prevent chatty client-server conversations, JEST provides a JSON encoder that supports circular references.

JEST's encoder for JSON introduces two additional properties, `$id` and `$ref`, that carry the persistent identity of the managed entities to resolve any cycle an object graph may have. The standard JSON parsers available in Dojo or Gson can interpret this enhanced representation, simply ignoring the additional `$id` and `$ref` properties. Only JEST can reinterpret the enhanced JSON representation to recreate an object graph with cyclic references.

Detached transactions: Key to unifying REST with JPA transactions

JPA's detached-transaction model is the key to meeting the challenges of REST's prescription for server-at-rest-between-requests semantics for transactional web applications. JEST integrates these two architectural styles by carrying out a transaction in the sequence shown in Figure 4:

Figure 4. Detached transaction for integrating REST and JPA



1. At time $t1$, the client request $R1$ for a resource reaches the server.

2. The server uses JPA to access the resource from the database as a set of Java objects. This access requires a database connection. The JPA provider releases the database connection once the access is complete.
3. The provider typically holds these objects in a short-term memory region called *persistent context* in JPA nomenclature. In this case, however, the provider immediately detaches the objects from the persistence context, converts them into a representation *A1* that is interpretable by the client, and sends *A1* as a response to original request *R1*. The immediate detachment ensures that the server application holds no resources after the response has been sent.
4. The client renders *A1* and lets the user modify the representation to *A1'*.
5. The client may send further requests *R2* and *R3* to access more resources and receives *A2* and *A3*, respectively, as responses.
6. The client may let the user modify *A2* or *A3* as well.
7. Once the user completes the modifications of all received responses, the client sends a request *R4* at time *t2* with all modified representations *A1'*, *A2'*, and *A3'*.
8. The server converts the received modified representations to Java objects and uses JPA to merge them into a persistence context. At this point, the JPA provider may require a database connection to validate that the changes are consistent — for example, to ensure that no other client has committed incompatible changes to the same objects contained in *A1'*, *A2'*, and *A3'* in the time between *t1* and *t2*. Eventually, if the changes are consistent and isolated, then the server commits the changes to the database via the JPA API.

In this REST-JPA unification scheme, the JPA application doesn't hold the detached persistent objects in short-term memory (persistence context) when their representations are being modified by the client. Nor does it require retaining a database connection to ensure the transactional warranty at a later time *t2*. The server remains stateless as far as database connection and short-term memory are concerned. The server requires these resources on an on-demand basis for a much shorter period than the time span between *t1* and *t2*, which is the total duration of the transaction from the client's perspective.

This unification scheme solves the critical problem of how a stateless server can support client transactions without holding any client context and without sacrificing an ACID transaction warranty.

JEST over HTTP

Idempotent versus transactional requests

In HTTP parlance, a request is either *idempotent* or *nonidempotent*. In the JEST context, the useful categorization of an HTTP request is whether the resultant JPA operation requires a database transaction or not. An idempotent operation is one that when executed more than once produces the same result for the same input. HTTP's GET verb is an example. Multiple client requests for a resource such as a static HTML page are expected to get the same page. In the JEST context — in which HTTP requests are effectively translated to JPA operations — no request, strictly speaking, is idempotent. Even two successive requests for `PurchaseOrder 2345` are not warranted to produce the same result, because in between the requests the status of `PurchaseOrder 2345` may change in the database. HTTP's PUT is also an idempotent operation. In the JEST context, two successive PUTs with identical payloads will generate a Duplicate Primary Key exception that causes the second request to fail.

The last JEST concept to understand is how it specifies a communication protocol and a URI scheme. REST never directly refers to any specific communication protocol between client and server. But, in practice, REST shares a close relationship with HTTP (and has greatly influenced the evolution of HTTP). Hence, the JEST project chose HTTP as JEST's communication protocol. (Following REST principles, JEST's core concepts — resources being persistent closures of managed entities, and representations being language-neutral and domain-agnostic — are independent of HTTP.)

JEST URI syntax

JEST interprets standard URIs to identify persistent resources. Standard URIs are defined in four segments: scheme, authority, path, and query (see [Resources](#)). JEST applies a special interpretation to the path and query segments to translate them to an appropriate persistence operation.

Here are a couple of typical JEST URIs that illustrate what information must be encoded in a URI for JEST to operate.

This URI gets an Actor with primary key m1:

```
http://openjpa.com:8080/jest/find/plan=basic?type=Actor&m1
```

This URI gets a single Actor with the name John via a Java Persistence Query Language (JPQL) query:

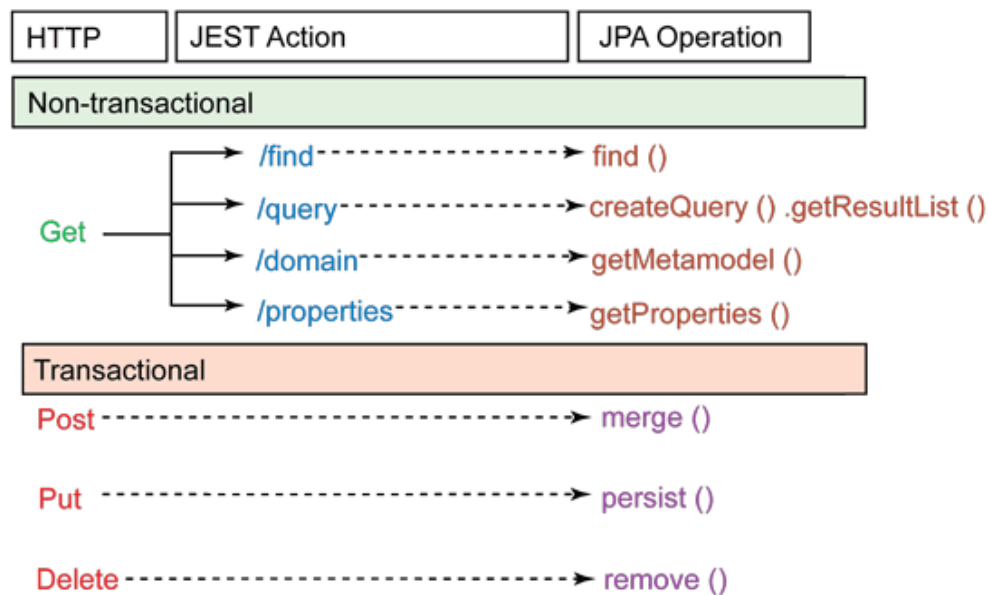
```
http://openjpa.com:8080/jest/query/single?q=select p from Actor p where p.name=:n&n=John
```

A JEST URI must encode sufficient information to furnish the following details:

- **JPA operation:** The persistent operations in the JEST context are mostly methods on the `javax.persistence.EntityManager` interface:
 - `find()`: Locate an entity by its primary identifier.
 - `persist()`: Create the given new entity in the database.
 - `merge()`: Update the current persistence context with the current state of the given entity.
 - `remove()`: Delete the given entity from the database.
- `EntityManager` also acts as a factory to create executable `javax.persistence.Query` instances from a JPQL query string.

The target persistence operation ideally should be implicit via an HTTP verb. The HTTP verbs PUT, POST, and DELETE directly translate to the corresponding JPA operation (`merge()`, `persist()`, and `remove()`, respectively) as shown in Figure 5:

Figure 5. Mapping HTTP verbs to JPA persistence operations



HTTP GET, however, must be multiplexed to a find or query operation. Besides these operations, HTTP GET is also used to get the metamodel (`/domain`) and configuration properties (`/properties`) of the persistence unit. To account for multiplicity of resource types available through GET, JEST requires that an action's name be encoded in the URI and mandates that the action be the first path segment following the

invocation context.

- **Qualifiers:** A persistence operation can be further qualified or decorated. For example, a query can be constrained to return one and only one result, or only the first 20 selected results. A find or query operation can be decorated to apply a fetch plan to customize the set of entities to fetch from the database.

The qualifiers are encoded in path segments subsequent to action. An action can have zero or more qualifiers. The order in which qualifiers appear is not significant. A qualifier appears as a key-value pair separated by the = character. For a qualifier with a boolean value, the value part can be omitted for brevity. Examples of boolean-valued qualifiers are `single` (to denote that a query must be satisfied by one and only one entity) or `named` (to denote that the specified query argument refers to the name of a declared `NamedQuery` rather than a JPQL string).

- **Arguments:** Each operation requires arguments. The arguments of a `find()` operation, for example, are the type of entity and its primary identifier. The arguments for a query operation are the JPQL string or moniker of a predefined named query. A parameterized query also takes a variable number of arguments as its binding parameters. The arguments to a JPA operation are encoded in the query portion of the URI for an HTTP GET request and in the payload for PUT, POST, and DELETE requests. Furthermore, the arguments encoded in the query portion of a GET-request URI depend on the action. For example, a `find()` operation takes two arguments: a Java class for the entity to be found and its persistent identifier.

Why the query segment for arguments?

JEST encodes JPA operation arguments in a GET-request URI's query segment because a JEST URI can be used to specify a JPQL query string. A JPQL query string can contain characters that are not permitted in a URI's path segment.

The query segment of the URI separates each argument by the ampersand (&) character, and each argument appears as a key-value pair separated by the equals (=) character. Unlike qualifiers, the arguments are ordered, and most of them are mandatory.

The critical aspect of converting a URI to an executable JPA operation is transferring string-based arguments to the strongly typed arguments that any JPA operation requires. For example, the argument `type=Actor` must be translated to `Actor.class`. Or a XML/JSON payload for a POST

request must be converted to a strongly typed Java object graph before being merged into the persistence context.

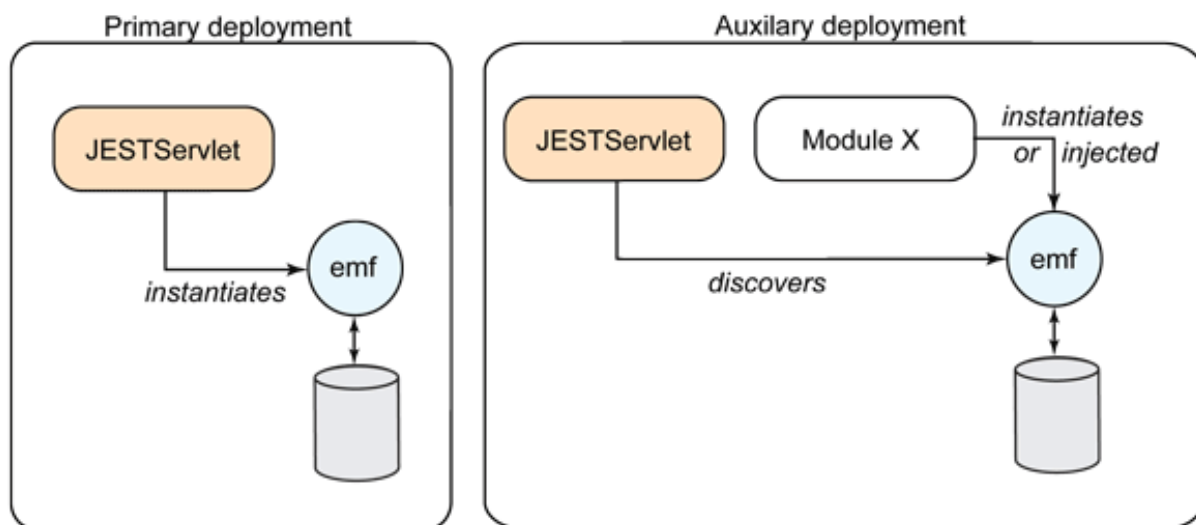
JEST in action

Now that you understand all of the conceptual and mechanical underpinnings of the JEST unification scheme, you're ready to take a look at a concrete implementation: `JESTServlet`. `JESTServlet` is a standard `javax.servlet.HttpServlet` for accessing JEST facilities from a remote client. A `JESTServlet` can be deployed in any standard servlet container, such as Tomcat, WebSphere®, or Glassfish.

Deploying JEST

`JESTServlet` is packaged with the OpenJPA installation. Download, build, and deployment instructions are available from the OpenJPA documentation site (see [Resources](#)), so I won't repeat those steps. Instead I'll describe the two deployment modes supported by `JESTServlet`: *primary* and *auxiliary*, as illustrated in Figure 6:

Figure 6. Primary and auxiliary modes of deploying JESTServlet



In primary mode, `JESTServlet` itself instantiates a persistence unit, `EntityManagerFactory`. The named persistence unit is described via a standard persistence descriptor: `META-INF/persistence.xml`, packaged in the web archive.

In auxiliary mode, `JESTServlet` does not have its own persistence unit but discovers a persistence unit of a *sibling component*: a deployed artifact such as another servlet in the same deployment module. In auxiliary deployment mode, `JESTServlet` needs to know the name of the persistence unit used by the sibling component. Currently, the sibling component must activate native pooling of OpenJPA's `EntityManagerFactory` so that `JESTServlet` can obtain a handle

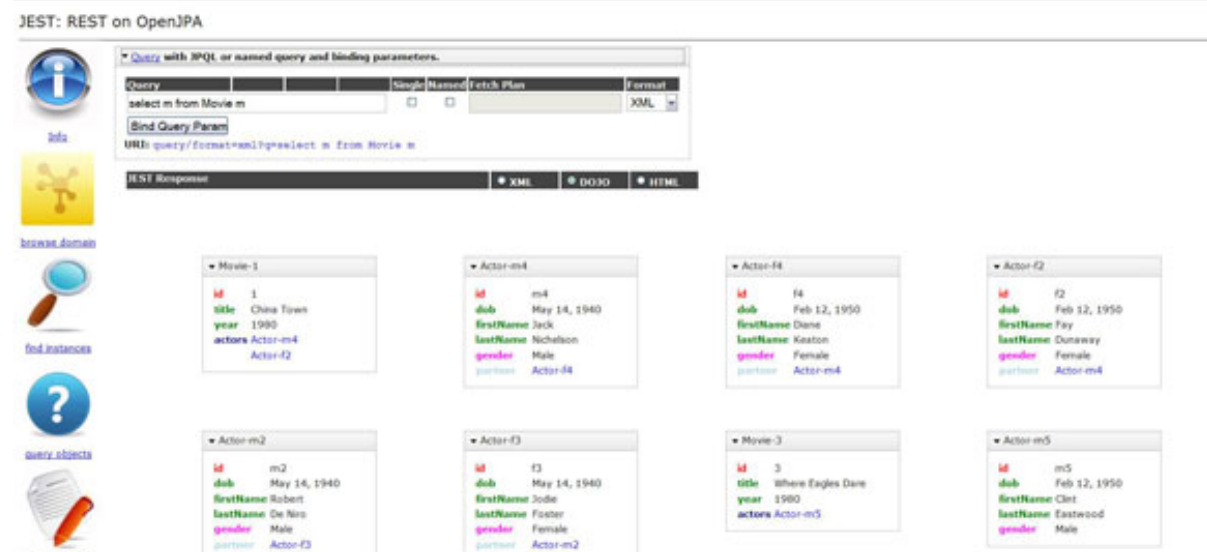
to the same unit from the pool. Different application and servlet containers may resort to proprietary mechanics to pool the persistence units (or their proxies) that the container often injects into the operational components via dependency injection. As of this writing, JESTServlet does not have a generalized mechanics to locate a pooled persistence unit from such container-managed pools, but the project is investigating addition of such a capability.

The prepackaged sample demonstrates the auxiliary mode whereby a separate simple servlet instantiates a persistence unit and is deployed alongside JESTServlet in the same web archive. The sample shows how JESTServlet, without any prior knowledge of the persistence unit of its sibling, can browse the persistence domain or represent the persistent instances via its generic, domain-agnostic capabilities.

A Dojo client for JEST

JEST deliberately does not address the issue of rendering a resource representation. For example, it does not provide a representation in HTML. JESTServlet, however, serves a single HTML page with embedded JavaScript. This page is effectively an Ajax client to JEST facilities. The client uses the Dojo JavaScript library to submit an asynchronous request for resources to JESTServlet using the [JEST URI syntax](#). It renders the response (in XML or JSON) either as it is received or in the form of Dojo visual widgets — validating my earlier assertion that specialized JSON encoder output accounting for cyclic graphs is consumable by standard JSON parsers. Figure 7 shows the page:

Figure 7. Dojo-based user interface to JEST



See the [full figure here](#).

This single, interactive web page makes all available JEST resources accessible to

the user: the persistent domain model, the persistent entities, and configuration of the persistence unit. It provides HTML forms for the user to build a request. These forms show how the JEST URI syntax is formed step-by-step by user-specified qualifiers and arguments.

JESTServlet executes each client request in a context that binds an HTTP servlet request-response to a persistence context. The lifetime of the context is same as the lifetime of the request-response cycle — thereby satisfying the stateless nature of REST in between requests.

Conclusion

JEST leverages the rich functional concepts of JPA architecture:

- A powerful query language
- Customizable closures
- Generic domain model
- Detached transactions to provide content-rich and dynamically configurable representation for remote, language-neutral clients with full compliance to REST architectural principles

JEST's approach is noninvasive to persistent resources, as opposed to other approaches with similar intent, such as JAX-RS. Because JEST is completely metadata-driven, it is a generic facility that applies to any persistent domain model without prior knowledge of its specifics.

JEST can be extended to represent the internal runtime state of OpenJPA — for example, the execution statistics of queries, or hit ratio of second-tier cache or cached entities. Access to such internal information could be useful for building a web-browser-based monitoring console.

An open issue for JEST is fine-grained data security. The provision to access persistent data as resources by a remote client raises data privacy and security concerns. The project is investigating how the content of a JEST response can be authenticated or controlled at a fine-grained data level based on the requesting client's credentials. Given that the JEST execution environment is aware of a persistent context and the expression tree of an executable query, a feasible approach is to predicate the query-expression tree nodes on a set of access rules based on a client's role. (The current JESTServlet does prevent any cross-browser scripting by rewriting the base URL of the JavaScript client on deployment.)

Resources

Learn

- [JEST](#): Explore the JEST website, including the pages on:
 - [JEST Representation](#): See how JEST represents a persistent Java object graph in XML or JSON.
 - [JEST usage](#): Read about the primary and auxiliary modes of deploying JESTServlet.
- [OpenJPA Documentation](#): Consult the official documentation for OpenJPA and JEST.
- [Representational State Transfer](#): Read Wikipedia's article on REST.
- [Architectural Styles and the Design of Network-based Software Architectures](#) (Roy Thomas Fielding, 2000): Fielding's Ph.D. dissertation originated REST concepts and terminology.
- ["Dynamic, typesafe queries in JPA 2.0"](#) (Pinaki Poddar, developerWorks, September 2009): Read about JPA 2.0's Metamodel API.
- [JSR 317: Java Persistence 2.0](#): Take a look at the current JPA specification.
- ["A brief introduction to REST"](#) (Stefan Tilkov, InfoQ, December 2007): Get an introduction to REST concepts.
- [Uniform Resource Identifiers \(URI\): Generic Syntax](#): This Request for Comments (RFC) defines the standard URI segments.
- [Dojo](#) and [Gson](#): Learn more about these two JavaScript libraries.
- [JSON](#): Read about JSON's lack of support for object references.
- [Transitive closure](#): Explore the concept behind persistence closures.
- [Fetch Groups](#): Read about OpenJPA's FetchPlan interface.
- Browse the [technology bookstore](#) for books on these and other technical topics.
- [developerWorks Java technology zone](#): Find hundreds of articles about every aspect of Java programming.

Get products and technologies

- [OpenJPA](#): Download OpenJPA.

Discuss

- Get involved in the [developerWorks community](#). Connect with other developerWorks users while exploring the developer-driven blogs, forums,

groups, and wikis.

About the author

Pinaki Poddar



Dr. Pinaki Poddar works in middleware technology with an emphasis on object persistence. He is a member of the Expert Group for the Java Persistence API (JSR 317) and Java Data Objects (JSR 12) specifications and a committer for the Apache OpenJPA project. In a past life, he contributed to the building component-oriented integration middleware for a global investment bank and a medical image-processing platform for healthcare industry.

Trademarks

Java and all Java-based trademarks are trademarks of Sun Microsystems, Inc. in the United States, other countries, or both.